Guía de uso — WaterManagement (WM)

Esta guía explica, paso a paso y sin dar nada por sabido, cómo poner en marcha y usar el proyecto de dos maneras:

- **Parte A — En tu propio PC (local):** para probar todo sin depender de internet ni de Railway.
- **Parte B — En Railway + laboratorio:** la forma "oficial" de la práctica (nube + PCs del laboratorio).

Al final hay un manual de uso del programa (qué se ve, qué botones hay, qué esperar) y una tabla de problemas frecuentes.

- **Parte C — En 3 ordenadores distintos:** orden exacto de encendido, qué corre en cada uno y cómo comprobar cada paso.

> Si solo quieres ver el proyecto funcionando: lee el apartado 1, haz la **Parte A** y termina en el apartado 5.
> Si vas a defender la práctica: **Parte B** (nube + laboratorio) y **Parte C** (el orden de encendido, resumido).

---

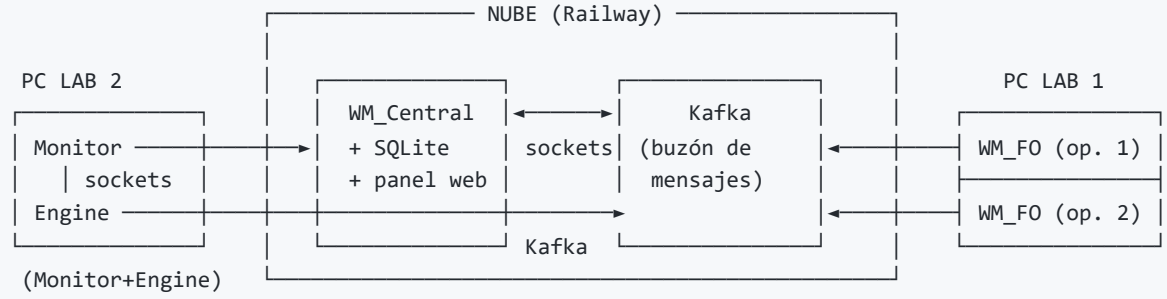
1. ¿Qué es este proyecto?

Es una **simulación de una red de riego urbano**. Hay tres tipos de programas:

Programa	Qué simula	Archivo
WM_Central (CENTRAL)	El "centro de control" del ayuntamiento. Guarda las estaciones en una base de datos y muestra un panel web con el estado de todas.	`WM_Central/WM_Central.py`
WM_WS (estación de riego)	Un punto de riego en un parque. Tiene dos partes: el **Monitor** (vigila la salud y se registra en Central) y el **Engine** (abre la válvula y mide el caudal).	`WM_WS/WM_WS_M.py` y `WM_WS/WM_WS_E.py`
WM_FO (operario de campo)	La aplicación del jardinero: ve las estaciones y pide que se active un riego.	`WM_FO/WM_FO.py`

Se comunican de dos maneras:

- **Sockets** (conexión directa): el Monitor habla con Central para registrarse, y el Engine habla con su Monitor para el "latido" de salud.
- **Kafka** (un "buzón" de mensajes en la nube): telemetría del riego, órdenes de Central, peticiones del operario y el estado general de la red.



Glosario mínimo

Palabra	Significa
WS	*Watering Station*, estación de riego.
FO	*Field Operator*, operario de campo.
Docker	Programa que ejecuta aplicaciones dentro de "cajas" (contenedores) iguales en cualquier PC.
Dockerfile	Receta que dice cómo construir la caja de un programa.
docker compose	Arranca varias cajas a la vez a partir de un archivo `.yml`.
Kafka	Sistema de mensajería: unos programas dejan mensajes en "topics" (canales) y otros los leen.
Railway	Servicio en la nube donde se ejecutan Central y Kafka.
TCP Proxy	Puerta pública que Railway abre para que algo de fuera (tu PC del laboratorio) llegue a un servicio.
Variable de entorno	Un ajuste (nombre = valor) que se le pasa a un programa sin tocar su código.

---

2. Qué necesitas

Programas

Programa	Para qué	Cómo comprobar que lo tienes
Docker Desktop	Arrancar Kafka y Central (local) y las estaciones en el laboratorio.	`docker --version` y `docker compose version`
Python 3.10 o superior	Ejecutar estaciones y operario en local sin Docker.	`python --version`
Git	Subir el código a GitHub.	`git --version`
VS Code (recomendado)	Editar el proyecto.	—

Abre una terminal (en Windows: *PowerShell* o *Símbolo del sistema*) y ejecuta los comandos de la tercera columna. Si alguno da "no se reconoce el comando", ese programa no está instalado.

> En Windows, Docker Desktop normalmente necesita permisos de administrador para instalarse. Si tu PC no los tiene, consúltalo con tu profesor: el enunciado exige Docker.

Cuentas (solo para la Parte B)

- Cuenta de **GitHub** (https://github.com).
- Cuenta de **Railway** (https://railway.com), creada preferiblemente con el login de GitHub.

---

3. Mapa del proyecto

```
Proyecto/
├── GUIA_USO.md              ← este archivo
├── README.md                ← resumen técnico
├── requirements.txt         ← librerías de Python (kafka-python-ng, Flask)
├── docker-compose.yml       ← arranca Kafka + Central (para PROBAR EN LOCAL; Railway no lo usa)
├── docker-compose.lab.yml   ← arranca estaciones y operario (PCs del laboratorio)
├── .env.lab.example         ← plantilla con los datos de Railway para el laboratorio
├── .gitignore               ← archivos que NO se suben a GitHub
├── utils/                   ← código compartido: protocol.py (tramas), kafka_utils.py, db.py
├── WM_Central/              ← Central: WM_Central.py, dashboard.py, templates/dashboard.html, Dockerf
├── WM_WS/                   ← Estación: WM_WS_M.py (Monitor), WM_WS_E.py (Engine), Dockerfile.monitor
├── WM_FO/                   ← Operario: WM_FO.py, Dockerfile, activaciones_ejemplo.json
└── scripts/                 ← lanzadores .bat de Windows para ejecutar SIN Docker
```

Puertos que se usan

Puerto	Quién lo usa	Para qué
5000	WM_Central	Recibe el registro de los Monitores (sockets).
8080	WM_Central	Panel web (en Railway se toma de la variable `PORT`).
9092	Kafka	Uso interno (entre contenedores / dentro de Railway).
9093	Kafka	Uso externo (estaciones y operarios que están fuera).
6001, 6002…	Monitor de cada estación	Donde el Monitor espera a su Engine. Cada estación usa uno distinto.

PARTE A — Probarlo todo en tu PC (local)

Usaremos Docker para la "nube" (Kafka + Central) y Python para las estaciones y el operario. Es la forma más sencilla de probar.

A.1 Preparar la carpeta

1. Descomprime el proyecto en una carpeta, por ejemplo `C:\Proyecto`.
2. Abre una terminal **dentro de esa carpeta**. En Windows: abre la carpeta en el Explorador, haz clic en la barra de direcciones, escribe `powershell` y pulsa Enter.
3. Comprueba que estás bien situado: el comando `dir` (Windows) o `ls` (Linux/Mac) debe mostrar `docker-compose.yml`, `WM_Central`, `utils`...

A.2 Arrancar Kafka y Central

1. Abre **Docker Desktop** y espera a que indique que está en marcha.
2. En la terminal, dentro de la carpeta del proyecto:

`docker compose up --build`

1. La primera vez descarga imágenes y tarda unos minutos. Verás muchas líneas de texto. **Deja esta terminal abierta**: es el "servidor".
2. Está listo cuando veas en la salida líneas parecidas a:

`wm_central | [WM_CENTRAL] Servidor de sockets escuchando en el puerto 5000... wm_central | [KAFKA] Escuchando estado de WS en 'estado-ws'...`

Kafka tarda un poco en estar listo; Central espera a que lo esté, así que no te preocupes si tarda un minuto.

1. Abre el navegador en **http://localhost:8080**. Verás el panel "WM_Central — Panel de monitorización", todavía **sin estaciones** (aún no hay ninguna conectada).

A.3 Preparar Python (solo la primera vez)

Abre una **segunda terminal** en la carpeta del proyecto.

Windows:

```
scripts\setup_windows.bat
```

Crea una carpeta `.venv` con un entorno de Python propio e instala las librerías.

Linux / Mac:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

A.4 Arrancar una estación de riego (Monitor + Engine)

Una estación son **dos programas** que se ejecutan cada uno en su terminal.

Windows (un solo comando)

```
scripts\run_ws_windows.bat 127.0.0.1 5000 WS-01 "River Park" 6001 127.0.0.1:9093
```

Se abren dos ventanas: *Monitor WS-01* y *Engine WS-01*.

Linux / Mac (dos terminales, con el entorno activado: `source .venv/bin/activate`)

Terminal 1 — Monitor:

```
python WM_WS/WM_WS_M.py 127.0.0.1 5000 WS-01 "River Park" 6001 127.0.0.1:9093
```

Terminal 2 — Engine (esperar 2 segundos a que el Monitor esté listo):

```
python WM_WS/WM_WS_E.py 127.0.0.1 6001 127.0.0.1:9093
```

Qué significan los datos del comando del Monitor

Dato	Ejemplo	Significa
1	`127.0.0.1`	Dirección de Central (en local, tu propio PC).
2	`5000`	Puerto de sockets de Central.
3	`WS-01`	Identificador de la estación. Tiene que ser único.
4	`"River Park"`	Ubicación (parque/jardín). Si lleva espacios, entre comillas.
5	`6001`	Puerto donde el Monitor espera a su Engine.
6	`127.0.0.1:9093`	Dirección de Kafka (en local, puerto 9093).

El Engine recibe: dirección del Monitor, puerto del Monitor y dirección de Kafka.

Qué debes ver

- Monitor: `Conectado a CENTRAL…` y `Respuesta de CENTRAL: REGISTRO_OK#WS-01`.
- En el panel web (http://localhost:8080) aparece **WS-01 – River Park en gris (DESCONECTADA)**: Central ya la conoce, pero todavía no sabe si puede regar.
- Cuando arranca el **Engine** (unos segundos después), pasa a **verde (DISPONIBLE)**.

> **Arranca siempre el Engine justo después de su Monitor.** Una estación solo se pone verde cuando Monitor *y* Engine están conectados.

Segunda estación (opcional)

Repite con otro identificador y otro puerto de Engine:

```
scripts\run_ws_windows.bat 127.0.0.1 5000 WS-02 "Riverside Gardens" 6002 127.0.0.1:9093
```

A.5 Arrancar un operario (FO)

En otra terminal con el entorno activado:

Windows:

```
scripts\run_fo_windows.bat 127.0.0.1:9093 FO-01
```

Linux / Mac:

```
python WM_FO/WM_FO.py 127.0.0.1:9093 FO-01
```

Aparece el menú del operario (ver apartado 5.3). Para un segundo operario, ejecuta lo mismo con `FO-02`.

A.6 Parar y limpiar

Quieres...	Haz esto
Cerrar un Monitor/Engine/FO	Cierra su ventana o pulsa `Ctrl + C`.
Parar Kafka y Central	En la terminal de `docker compose up`, pulsa `Ctrl + C`; después `docker compose down`.
Borrar todos los datos (estaciones, mensajes) y empezar de cero	`docker compose down -v`

A.7 Variante: todo en Docker en el mismo PC

Si quieres usar Docker también para las estaciones y el operario en tu PC, **no uses** `127.0.0.1`: desde dentro de un contenedor eso apunta al propio contenedor, no a Kafka. Usa la **IP de tu PC en la red** y sigue el **Escenario C2** de la Parte C, haciendo que los tres "ordenadores" sean el mismo.

PARTE B — Railway + laboratorio

Esquema del despliegue obligatorio:

- **Railway (nube):** Kafka + WM_Central (+ su base de datos SQLite).
- **PC LAB 2:** estaciones de riego (Monitor + Engine) en Docker.
- **PC LAB 1:** operarios de campo (WM_FO) en Docker.

B.1 Subir el código a GitHub

1. En GitHub: **New repository**, ponle nombre (por ejemplo `WaterManagement-SD`) y créalo vacío.
2. En la carpeta del proyecto, abre una terminal y ejecuta (cambia la URL por la de tu repo):

`git init git add . git commit -m "Primera version" git branch -M main git remote add origin https://github.com/TU_USUARIO/TU_REPO.git git push -u origin main`

(Si ya tienes el repositorio con historial, simplemente copia los archivos nuevos encima y haz `git add .`, `git commit` y `git push`.)

1. Comprueba que **no** se han subido `.env`, `.env.lab` ni archivos `.db` (el `.gitignore` los excluye).
2. **Dar acceso al profesor:** en GitHub, entra en *Settings* → *Collaborators* → *Add people* y añade a tu profesor de prácticas.

B.2 Crear el proyecto en Railway

1. Entra en https://railway.com/dashboard, pulsa **New Project**.
2. Se creará un proyecto vacío. Aquí añadirás dos servicios: `kafka` y `wm_central`.

> Railway **no usa** el archivo `docker-compose.yml`. Cada servicio se crea a mano, como se explica a continuación.

B.3 Servicio Kafka

> **Si ya tienes Kafka funcionando en Railway, no repitas este apartado.** Es la receta para crearlo desde cero, basada en el `docker-compose.yml` del proyecto.

B.3.1 Crear el servicio

1. En el proyecto: **New** → **Docker Image** y escribe:
   `bitnamilegacy/kafka:4.0.0-debian-12-r10`
2. Cambia el nombre del servicio a `kafka` (en minúsculas; otros pasos lo referencian por ese nombre).

B.3.2 Volumen (para que los datos persistan)

1. Dentro del servicio, añade un **Volume** (clic derecho sobre el servicio en el lienzo, o desde su pestaña de ajustes).
2. **Mount path:** `/bitnami/kafka`.

B.3.3 Puerta pública (TCP Proxy)

1. En el servicio `kafka`: **Settings** → **Networking** → **TCP Proxy**.
2. Escribe el puerto **9093** y confirma.
3. Railway mostrará una dirección tipo `algo.proxy.rlwy.net:12345`. Esa será la dirección pública de Kafka.

B.3.4 Variables

En el servicio `kafka`, pestaña **Variables** → **Raw Editor**, pega esto:

```
RAILWAY_RUN_UID=0
ALLOW_PLAINTEXT_LISTENER=yes
KAFKA_CFG_NODE_ID=1
KAFKA_CFG_PROCESS_ROLES=controller,broker
KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1@127.0.0.1:9094
KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER
KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,EXTERNAL://:9093,CONTROLLER://:9094
KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://${{RAILWAY_PRIVATE_DOMAIN}}:9092,EXTERNAL://${{RAILWAY_TCP_PI
KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT
KAFKA_CFG_INTER_BROKER_LISTENER_NAME=PLAINTEXT
KAFKA_CFG_OFFSETS_TOPIC_REPLICATION_FACTOR=1
KAFKA_CFG_TRANSACTION_STATE_LOG_REPLICATION_FACTOR=1
KAFKA_CFG_TRANSACTION_STATE_LOG_MIN_ISR=1
KAFKA_CFG_LOG_RETENTION_MINUTES=60
KAFKA_CFG_LOG_ROLL_MS=600000
KAFKA_CFG_LOG_RETENTION_CHECK_INTERVAL_MS=60000
```

Qué hace cada grupo:

Variable(s)	Para qué sirve
`RAILWAY_RUN_UID=0`	Railway monta los volúmenes como administrador; esta variable evita el error "Permission denied" al escribir en el volumen.
`KAFKA_CFG_NODE_ID`, `PROCESS_ROLES`, `CONTROLLER_*`	Kafka en modo KRaft (un solo nodo, sin Zookeeper).
`KAFKA_CFG_LISTENERS`	Los puertos en los que escucha.
`KAFKA_CFG_ADVERTISED_LISTENERS`	La dirección que Kafka **le dice a los clientes** que usen. Dentro de Railway usa el nombre privado; fuera usa la dirección del TCP Proxy. **Si esta variable está mal, los clientes conectan una vez y después fallan.**
`*_REPLICATION_FACTOR`, `MIN_ISR`	Necesarios al ser un solo nodo.
`KAFKA_CFG_LOG_*`	Mantienen el volumen pequeño: borran mensajes de más de 1 hora. Sin ellas el volumen crece hasta llenarse.

> `${{RAILWAY_TCP_PROXY_DOMAIN}}` solo tiene valor si ya hiciste el paso B.3.3 (TCP Proxy). Por eso se hace antes de las variables.

Pulsa **Deploy** y espera a que termine.

B.3.5 Comprobar

En **Deployments** → **View Logs** del servicio `kafka` debe aparecer, tras unos segundos, una línea que indica que el servidor Kafka ha arrancado, y no debe haber un error repetido.

B.4 Servicio WM_Central

B.4.1 Crear el servicio

1. En el proyecto: **New** → **GitHub Repo** y elige tu repositorio (la primera vez Railway pide permiso para acceder a tus repositorios de GitHub).
2. Cambia el nombre del servicio a `wm_central`.
3. **No pongas "Root Directory"**: déjalo vacío (el Dockerfile necesita ver toda la carpeta del proyecto, incluido `utils/`).

B.4.2 Variables

Pestaña **Variables** → **Raw Editor**:

```
RAILWAY_DOCKERFILE_PATH=WM_Central/Dockerfile
KAFKA_BROKER=${{kafka.RAILWAY_PRIVATE_DOMAIN}}:9092
SOCKET_PORT=5000
PORT=8080
DB_PATH=/app/data/wm_central.db
```

Variable	Para qué sirve
`RAILWAY_DOCKERFILE_PATH`	Indica a Railway qué Dockerfile usar (el suyo no está en la raíz). Si no lo encuentra, prueba con una barra inicial: `/WM_Central/Dockerfile`.
`KAFKA_BROKER`	Dónde está Kafka. Usa el nombre privado de Railway (`kafka` es el nombre del servicio del paso anterior, y distingue mayúsculas).
`SOCKET_PORT`	Puerto donde Central espera a los Monitores.
`PORT`	Puerto del panel web. Debe coincidir con el puerto que le pondrás al dominio (8080). **Nunca pongas 5000 aquí**, chocaría con los sockets.
`DB_PATH`	Dónde se guarda la base de datos SQLite.

> Si Central no consigue conectar con Kafka por la dirección privada, como alternativa pon en `KAFKA_BROKER` la dirección **pública** de Kafka (la del TCP Proxy, `dominio:puerto`).

B.4.3 Volumen (para no perder la base de datos)

Añade un **Volume** al servicio `wm_central` con **Mount path** `/app/data`. Sin él, la base de datos se borra en cada despliegue.

B.4.4 Dominio del panel web

1. **Settings** → **Networking** → **Generate Domain**.
2. Si pregunta por el puerto, escribe **8080**.
3. Obtendrás algo como `https://wm-central-production.up.railway.app`. Esa es la dirección del panel.

B.4.5 Puerta de los sockets (TCP Proxy)

1. En la misma sección **Networking** → **TCP Proxy**, escribe el puerto **5000**.
2. Railway mostrará algo como `nozomi.proxy.rlwy.net:31147`. Esa es la dirección de Central para las estaciones.

B.4.6 Comprobar

1. **Deployments** → **View Logs**: debe aparecer el banner con `WM_Central - WaterManagement`, `Puerto sockets : 5000` y `Puerto web : 8080`, seguido de `Servidor de sockets escuchando en el puerto 5000`.
2. Abre la URL del dominio en el navegador: debe verse el panel (sin estaciones, por ahora).

B.5 Anotar los tres datos para el laboratorio

Dato	De dónde sale	Ejemplo
`CENTRAL_HOST`	Dominio del TCP Proxy de `wm_central`, **sin el puerto**	`nozomi.proxy.rlwy.net`
`CENTRAL_PORT`	Puerto del TCP Proxy de `wm_central`	`31147`
`KAFKA_BROKER`	Dirección del TCP Proxy de `kafka`, **con puerto**	`algo.proxy.rlwy.net:12345`

B.6 PC LAB 2 — Estaciones de riego (Docker)

1. Descarga el proyecto del repositorio de GitHub (`git clone https://github.com/TU_USUARIO/TU_REPO.git`) o copia la carpeta.
2. Entra en la carpeta del proyecto con una terminal.
3. Crea el archivo de datos del laboratorio a partir de la plantilla:
4. Windows (PowerShell): `copy .env.lab.example .env.lab`
5. Linux/Mac: `cp .env.lab.example .env.lab`
6. Abre `.env.lab` con un editor y sustituye los tres valores por los de B.5. **Sin `http://`, sin comillas, y el dominio sin el puerto**:

```
CENTRAL_HOST=nozomi.proxy.rlwy.net CENTRAL_PORT=31147 KAFKA_BROKER=algo.proxy.rlwy.net:12345
```

1. Arranca los **Monitores** (quedan en segundo plano):

```
docker compose -f docker-compose.lab.yml --env-file .env.lab up --build -d ws01_monitor ws02_monitor
```

1. Arranca cada **Engine** en su propia terminal (necesitan teclado para simular fugas):

```
docker compose -f docker-compose.lab.yml --env-file .env.lab run --rm ws01_engine
docker compose -f docker-compose.lab.yml --env-file .env.lab run --rm ws02_engine
```

1. Mira el panel web de Railway: **WS-01 (River Park)** y **WS-02 (Riverside Gardens)** aparecen en verde.

Los datos de cada estación (id, ubicación, puerto) están escritos en `docker-compose.lab.yml`; si quieres más estaciones, copia un bloque monitor + engine y cambia `WS_ID`, `WS_UBICACION`, `ENGINE_PORT` / `MONITOR_PORT` (puerto distinto en cada una) y los nombres.

B.7 PC LAB 1 — Operarios de campo (Docker)

1. En el otro PC, repite los pasos 1–4 del apartado B.6 (descargar el proyecto y crear `.env.lab`). El operario solo usa `KAFKA_BROKER`.
2. Arranca el operario 1:

```
docker compose -f docker-compose.lab.yml --env-file .env.lab run --rm fo1
```

1. Para un segundo operario, en otra terminal:

```
docker compose -f docker-compose.lab.yml --env-file .env.lab run --rm -e OPERADOR_ID=FO-02 fo1
```

B.8 Actualizar el despliegue (CI/CD)

El ciclo es: **VS Code → GitHub → Railway**.

1. Cambias algo en el código.
2. `git add .` → `git commit -m "mensaje"` → `git push`.
3. Railway detecta el cambio en GitHub y reconstruye `wm_central` automáticamente.

Si en los ajustes del servicio ves el aviso **"Auto deploy unavailable. Could not load branches"**, pulsa **Retry** o revisa que Railway tenga acceso a tu repositorio en GitHub; si no, el push no desplegará solo.

Cambios en el laboratorio: `git pull` en el PC y repetir los comandos con `--build`.

B.9 Lista de comprobación antes de la evaluación

- [ ] Kafka en Railway funciona (logs sin errores repetidos).
- [ ] `wm_central` funciona y el panel web abre desde el navegador.
- [ ] Anotados `CENTRAL_HOST`, `CENTRAL_PORT` y `KAFKA_BROKER`.
- [ ] Volúmenes de Kafka (`/bitnami/kafka`) y Central (`/app/data`) creados.
- [ ] Variables de retención de Kafka puestas (para que el volumen no se llene).
- [ ] Probado el flujo completo (apartado 5.5) desde los PCs del laboratorio, siguiendo el orden de encendido de la Parte C1.
- [ ] Repositorio de GitHub con acceso para el profesor.
- [ ] Memoria con portada, descripción de componentes, esta guía de despliegue y capturas.
- [ ] Si quieres empezar con el volumen de Kafka vacío, hazlo **antes** de la evaluación (ver el apartado de problemas).

PARTE C — Encender el sistema en 3 ordenadores

Hay dos formas de repartirlo en tres sitios. La **C1** es la oficial de la práctica; la **C2** sirve para ensayar sin Railway con tres PCs de una misma red.

C1. Escenario oficial: Railway + PC LAB 2 + PC LAB 1

Sitio	Qué ejecuta	Qué necesita instalado	Archivo de datos
1. Railway (nube)	Kafka y WM_Central (con su base de datos)	Nada: lo ejecuta Railway	Variables de Railway
2. PC LAB 2	Estaciones de riego: Monitor + Engine (WS-01, WS-02…)	Docker Desktop y el proyecto	`.env.lab`
3. PC LAB 1	Operarios de campo (WM_FO)	Docker Desktop y el proyecto	`.env.lab`

Antes de empezar (una sola vez)

- La parte de Railway está creada y desplegada (apartados B.2 a B.5).
- Tienes anotados `CENTRAL_HOST`, `CENTRAL_PORT` y `KAFKA_BROKER` (apartado B.5).
- En PC LAB 2 y PC LAB 1: el proyecto descargado y `.env.lab` creado y rellenado (apartado B.6, pasos 1–4).

Orden de encendido (respétalo)

Paso	Dónde	Qué haces	Qué debes ver antes de seguir
1	Railway	Comprueba que el servicio `kafka` está en verde. Si lo reinicias, espera a que acabe.	Logs de `kafka` sin errores repetidos.
2	Railway	Comprueba que `wm_central` está en verde (si acabas de cambiar algo, *Redeploy*).	Logs: `Servidor de sockets escuchando en el puerto 5000`.
3	Tu navegador (cualquiera)	Abre la URL del panel (dominio de `wm_central`).	El panel se abre. Todas las estaciones, si las hay, en **gris**.
4	PC LAB 2	`docker compose -f docker-compose.lab.yml --env-file .env.lab up --build -d ws01_monitor ws02_monitor`	`docker compose -f docker-compose.lab.yml logs ws01_monitor` muestra `Respuesta de CENTRAL: REGISTRO_OK#WS-01`. En el panel, WS-01 y WS-02 en **gris**.
5	PC LAB 2 (una terminal por Engine)	`docker compose -f docker-compose.lab.yml --env-file .env.lab run --rm ws01_engine` y, en otra terminal, `… run --rm ws02_engine`	En el panel, WS-01 y WS-02 pasan a **verde**.
6	PC LAB 1	`docker compose -f docker-compose.lab.yml --env-file .env.lab run --rm fo1` (otro operario: `… run --rm -e OPERADOR_ID=FO-02 fo1`)	Aparece el menú del operario. La opción 1 lista las estaciones como DISPONIBLE.
7	PC LAB 1	Opción 2 del menú: `WS-01`, duración `20`.	El panel pone WS-01 en **verde parpadeando** con caudal, volumen y `FO-01`.

Si en algún paso no ves lo esperado, **no sigas**: ve al apartado 6 (problemas frecuentes).

Orden de apagado

1. PC LAB 1: opción 4 (salir) en cada operario.
2. PC LAB 2: `Ctrl + C` en cada Engine y después `docker compose -f docker-compose.lab.yml down`.
3. Railway: no hace falta apagar nada. Las estaciones pasarán a gris en unos 15 s y se quedarán así hasta la próxima vez.

Si algo se reinicia a mitad

Qué se reinicia	Qué pasa	Qué haces
`wm_central` (Redeploy)	Todas las estaciones se ponen en gris al arrancar y vuelven solas a su estado real en unos segundos.	Nada. Espera 10–20 s.
`kafka`	Se pierden los mensajes en vuelo. Central, estaciones y operarios pueden quedarse sin recibir mensajes.	Después de Kafka, reinicia `wm_central`, luego las estaciones y luego los operarios (en ese orden).
Un **Monitor**	Su estación pasa a gris en ≤ 15 s.	Arráncalo de nuevo y, después, su Engine.
Un **Engine**	Su estación pasa a gris.	Arráncalo de nuevo.

C2. Escenario de ensayo: 3 PCs de una misma red, sin Railway

Sirve para probar el reparto en tres máquinas aunque no tengas la nube a mano. Todas deben estar en la misma red (mismo router o cable).

PC	Rol	Qué ejecuta
PC-A	"La nube"	Kafka + WM_Central (con `docker-compose.yml`)
PC-B	Estaciones	Monitor + Engine (con `docker-compose.lab.yml`)
PC-C	Operarios	WM_FO (con `docker-compose.lab.yml`)

> Las redes de algunas universidades y los Wi-Fi públicos aíslan los equipos entre sí y esto no funcionará. Usa un router propio, cable o el punto de acceso del móvil.

C2.1 En PC-A

1. **Averigua su IP en la red.**
2. Windows: abre una terminal y escribe `ipconfig`. Busca "Dirección IPv4" del adaptador que usas (p. ej. `192.168.1.50`).
3. Linux/Mac: `hostname -I` (Linux) o `ipconfig getifaddr en0` (Mac).
   Esta guía usará `192.168.1.50`: **sustitúyela por la tuya** en todos los comandos.
4. **Abre los puertos en el cortafuegos** (Windows, PowerShell como administrador):
   `New-NetFirewallRule -DisplayName "WM puertos" -Direction Inbound -Protocol TCP -LocalPort 5000,8080,9093 -Action Allow`
5. **Crea un archivo llamado** `.env` en la raíz del proyecto (con VS Code: *New File*, nombre `.env`; ojo con que no quede como `.env.txt`) con una sola línea:
   `KAFKA_EXTERNAL_HOST=192.168.1.50`
   Esto hace que Kafka se anuncie con esa IP para que los otros PCs puedan conectar.
6. Arranca: `docker compose up --build`
7. **Comprueba:** abre `http://192.168.1.50:8080` en el navegador de cualquier PC. Debe verse el panel.

C2.2 En PC-B (estaciones)

1. Copia `.env.lab.example` a `.env.lab` y ponle:
   `CENTRAL_HOST=192.168.1.50 CENTRAL_PORT=5000 KAFKA_BROKER=192.168.1.50:9093`
2. **Comprueba la conexión** (Windows PowerShell):
   `Test-NetConnection 192.168.1.50 -Port 9093 Test-NetConnection 192.168.1.50 -Port 5000`
   Ambos deben decir `TcpTestSucceeded : True`. Si dicen `False`: cortafuegos de PC-A o red aislada.
3. Arranca los Monitores y los Engines igual que en los pasos 4 y 5 de la tabla C1.

C2.3 En PC-C (operarios)

1. Copia `.env.lab.example` a `.env.lab` y ponle:
   `CENTRAL_HOST=192.168.1.50 CENTRAL_PORT=5000 KAFKA_BROKER=192.168.1.50:9093`
   (El operario solo usa `KAFKA_BROKER`; los otros dos valores no se usan, pero hay que rellenarlos para que el archivo esté completo.)
2. Arranca el operario como en el paso 6 de la tabla C1.

C2.4 Orden de encendido

PC-A (Kafka + Central) → esperar a ver el panel → PC-B (Monitores, después Engines) → PC-C (operarios).

C2.5 Cosas a tener en cuenta

- Si la IP de PC-A cambia (por ejemplo al reiniciar el router), actualiza `.env` en PC-A, reinicia (`docker compose down` y `up`) y actualiza `.env.lab` en PC-B y PC-C.
- Todos los comandos de PC-B y PC-C son idénticos a los del laboratorio: lo único que cambia son los tres valores del `.env.lab`.

Resumen en una tabla (qué cambia entre escenarios)

Dato	Local en un solo PC (Parte A)	C1: Railway + laboratorio	C2: 3 PCs en red
Dirección de Kafka	`127.0.0.1:9093`	`dominio-del-proxy:puerto`	`IP-de-PC-A:9093`
`CENTRAL_HOST`	`127.0.0.1`	dominio del TCP Proxy de `wm_central`	`IP-de-PC-A`
`CENTRAL_PORT`	`5000`	puerto del TCP Proxy	`5000`
Panel web	`http://localhost:8080`	URL del dominio de Railway	`http://IP-de-PC-A:8080`
Quién arranca Kafka y Central	`docker compose up`	Railway	`docker compose up` en PC-A

5. Manual de uso del programa

5.1 El panel web de Central

Se abre en el navegador (local: `http://localhost:8080`; Railway: la URL del dominio). Se actualiza solo cada segundo.

- **Arriba:** contadores por estado (Disponible, Regando, Fuga, Fuera de servicio, Desconectada).
- **Tabla de estaciones:** ID, ubicación, estado, caudal (L/min), volumen acumulado (L) y operario.
- **Eventos del sistema:** lista de lo último que ha pasado (registros, fugas, riegos, bloqueos…).

Estados y colores

Estado	Color	Significa
DISPONIBLE	Verde	La estación funciona y espera una petición de riego.
REGANDO	Verde parpadeando	Está regando. Se ve caudal, volumen acumulado y el operario que lo pidió.
FUGA	Rojo	Hay una fuga detectada: no se puede regar hasta resolverla.
FUERA DE SERVICIO	Naranja	Central la ha bloqueado a propósito.
DESCONECTADA	Gris	La estación no está conectada a Central.

Importante:
- Al arrancar Central, **se borra todo lo que sabía del estado** de las estaciones guardadas (estado, caudal, volumen y operario). Todas salen **DESCONECTADAS** (gris) hasta que cada una conecte de nuevo. Se conservan su ID y su ubicación.
- Una estación pasa a su estado real (verde, o rojo si tiene una fuga) cuando su Monitor se registra **y** su Engine está conectado. Si Central se reinicia con las estaciones encendidas, las recupera solas en unos segundos.
- Si cierras el Monitor de una estación (o se corta la red), pasa a gris en unos 15 segundos.
- No se puede **Bloquear** ni **Activar** una estación desconectada: el panel avisa con un mensaje.

Botones de cada fila

Botón	Qué hace
Regar	Pide la duración en segundos (60 por defecto) e inicia un riego, como si lo pidiera un operario. Solo funciona si la estación está DISPONIBLE.
Bloquear	Detiene cualquier riego y pone la estación FUERA DE SERVICIO.
Activar	Quita el bloqueo y la deja DISPONIBLE.

5.2 Estación de riego (Monitor y Engine)

- **Monitor:** se registra en Central y, cada segundo, pregunta al Engine si está bien (`PING`). Si no hay respuesta o el Engine contesta KO, avisa de una **FUGA**. También mantiene un latido con Central para que sepa que sigue vivo.
- **Engine:** abre la "electroválvula" cuando Central lo ordena y envía el caudal y el volumen cada segundo. Escucha el teclado:

Tecla + Enter	Efecto
`k`	Simula una **fuga** (el Engine contesta KO). La estación pasa a rojo.
`o`	Resuelve la avería. La estación vuelve a verde.
`q`	Cierra el Engine.

Si hay una fuga durante un riego, el riego se corta inmediatamente y el operario recibe el aviso.

5.3 Operario de campo (WM_FO)

Al arrancar muestra este menú:

```
===== WM_FO - Menu del operario =====
1) Ver estaciones (disponibles y su estado)
2) Solicitar activacion manual de riego
3) Ejecutar activaciones desde fichero
4) Salir
```

Opción	Qué hace
1	Muestra la lista de estaciones y su estado. Si dice que aún no hay información, espera unos segundos (Central la publica cada 2 s).
2	Pide el ID de la estación (ej. `WS-01`) y la duración en segundos (por defecto 60). Envía la petición a Central, que la autoriza o la deniega, y muestra los pasos y el resumen final.
3	Lee un archivo JSON con una lista de riegos y los pide uno a uno, esperando 4 segundos entre cada uno.
4	Sale.

El operario solo recibe los mensajes de las estaciones que le están dando servicio.

Formato del fichero de activaciones

`WM_FO/activaciones_ejemplo.json`:

```
[
  { "id_ws": "WS-01", "duracion_seg": 20 },
  { "id_ws": "WS-02", "duracion_seg": 15 },
  { "id_ws": "WS-01", "duracion_seg": 10 }
]
```

Cada elemento es una estación y cuántos segundos regar. Con Docker, la ruta ya viene en `ACTIVACIONES_FILE`; sin Docker puedes pasarla como tercer dato al arrancar o escribirla en la opción 3.

5.4 Qué hace Central cuando recibe una petición de riego

1. Comprueba que la estación existe y está **DISPONIBLE** (no regando, sin fuga, no bloqueada, conectada).
2. Si no lo está, **deniega** y dice el motivo.
3. Si lo está, ordena al Engine abrir la válvula y avisa al operario (**AUTORIZADO**).
4. Durante el riego recibe caudal y volumen cada segundo y los muestra en el panel.
5. Al terminar (por tiempo o por una fuga) envía al operario un **resumen**: volumen total y duración.

5.5 Prueba guiada de punta a punta

Con Central, Kafka, una estación y un operario arrancados:

1. **Panel:** WS-01 aparece en **gris** al arrancar su Monitor y pasa a **verde** al arrancar su Engine. *(Si no, ve al apartado 6.)*
2. **FO, opción 1:** ves WS-01 como DISPONIBLE.
3. **FO, opción 2:** `WS-01`, duración `20`. En el panel, WS-01 pasa a **verde parpadeando** con caudal, volumen y el operario `FO-01`.
4. **Fuga:** en la terminal del Engine escribe `k` + Enter. El panel pone WS-01 en **rojo** y el riego se corta; el FO avisa de que se interrumpió.
5. **Resolver:** escribe `o` + Enter. WS-01 vuelve a **verde**.
6. **Bloquear:** en el panel, botón **Bloquear** de WS-01 → pasa a **naranja** (FUERA DE SERVICIO). Intenta regar desde el FO: lo **deniega**. Pulsa **Activar** y vuelve a verde.
7. **Desconexión:** cierra el Monitor de WS-01. En unos segundos pasa a **gris** (DESCONECTADA). Vuélvelo a arrancar y vuelve a verde.
8. **Reinicio de Central:** reinícala (en Railway: Redeploy). Al arrancar, todas aparecen en gris y vuelven a verde cuando cada Monitor se reconecta.
9. **Fichero:** en el FO, opción 3 con el fichero de ejemplo: pide los riegos uno tras otro, con 4 s de pausa.

---

6. Problemas frecuentes

Síntoma	Causa probable	Solución
Aparece `Broker no disponible aun (x/10)`	Kafka aún no ha arrancado, o la dirección es incorrecta.	Espera 1–2 minutos. Revisa la dirección de Kafka (`127.0.0.1:9093` en local; `dominio:puerto` del proxy en el laboratorio). Comprueba que Kafka está en marcha (`docker compose ps` o los logs en Railway).
Conecta a Kafka una vez y luego da errores o se queda colgado	`KAFKA_CFG_ADVERTISED_LISTENERS` incorrecto (Kafka anuncia una dirección que el cliente no puede alcanzar).	Revisa B.3.4: la parte `EXTERNAL://…` debe ser el dominio y puerto públicos del TCP Proxy de Kafka. Tras corregirlo, redespliega Kafka.
En Railway, el panel da error 502 ("Application failed to respond")	El puerto del dominio no coincide con el que escucha la aplicación.	Define `PORT=8080` en `wm_central`, y que el dominio apunte al puerto 8080 (Settings → Networking). Redespliega.
Kafka en Railway se reinicia con `Permission denied`	Los volúmenes se montan como administrador.	Añade `RAILWAY_RUN_UID=0` al servicio `kafka` y redespliega.
Railway no encuentra el Dockerfile	Falta o está mal la variable.	`RAILWAY_DOCKERFILE_PATH=WM_Central/Dockerfile` (o con barra inicial). Y sin "Root Directory".
La estación sale **gris** aunque el Monitor está en marcha	Falta arrancar su **Engine**: sin Engine la estación no puede regar y Central la mantiene desconectada.	Arranca el Engine (apartado A.4 o B.6).
La estación no aparece / se queda **gris**	El Monitor no consigue registrarse en Central.	Mira los logs del Monitor: si dice `No se pudo registrar en CENTRAL…`, revisa `CENTRAL_HOST` y `CENTRAL_PORT` (dominio del TCP Proxy de Central, sin `http://` y sin el puerto en el host). Comprueba que el TCP Proxy de Central apunta al puerto 5000.
Todas las estaciones salen grises tras reiniciar Central	Es lo esperado: Central no conoce el estado real hasta que cada Monitor conecte.	Espera unos segundos: los Monitores se vuelven a registrar solos.
Una estación sigue naranja ("Fuera de servicio")	Se pulsó **Bloquear** en el panel.	Pulsa **Activar**.
Sale un aviso "La estación está desconectada" al pulsar Bloquear/Activar/Regar	La estación no está conectada: Central no puede darle órdenes.	Arranca su Monitor y su Engine y repite.
El FO dice "Aún no se ha recibido	Aún no llegó el primer mensaje, o el FO no llega a Kafka.	Espera unos segundos. Si persiste, revisa la dirección de Kafka del FO.

Síntoma	Causa probable	Solución
información de CENTRAL"		
Las teclas `k` / `o` no hacen nada en un Engine en Docker	El contenedor no tiene consola interactiva.	Arráncalo con `docker compose … run --rm ws01_engine` (no con `up -d`).
Error `cannot import name 'classproperty'`	Están instaladas a la vez `kafka-python` y `kafka-python-ng`.	`pip uninstall kafka-python` y después `pip install kafka-python-ng`.
Error `No se encontro utils/`	Ejecutas los programas desde otra carpeta.	Ejecútalos desde la **raíz** del proyecto.
`port is already allocated` / puerto en uso (5000, 8080, 9092, 9093)	Otro programa usa ese puerto.	Ciérralo, o cambia el puerto en `.env` (`SOCKET_PORT`, `WEB_PORT`).
`docker compose up` se queda esperando a Kafka	Kafka aún no está "healthy".	Espera; mira `docker compose logs kafka`.
El volumen de Kafka en Railway se llena	Sin retención, Kafka acumula mensajes.	Pon las tres variables `KAFKA_CFG_LOG_*` (B.3.4). Para vaciarlo ahora: *Wipe volume* **solo del volumen de Kafka** (no el de `wm_central`), y después reinicia Kafka, luego `wm_central`, luego estaciones y operarios.
Railway no redespliega al hacer `git push`	Aviso "Auto deploy unavailable".	Pulsa **Retry** en los ajustes del servicio o revisa el acceso de Railway a tu repositorio de GitHub.
Quiero empezar de cero en local	—	`docker compose down -v` y vuelve a arrancar.

> **Cuidado con *Delete Volume*:** borra el volumen y detiene los despliegues del servicio. Para vaciar datos usa *Wipe volume*.

---

7. Referencia técnica

7.1 Argumentos de cada programa

Todos aceptan los valores por línea de comandos **o** por variable de entorno (la línea de comandos tiene prioridad).

Programa	Línea de comandos	Variables de entorno
WM_Central	`python WM_Central/WM_Central.py <puerto_sockets> <kafka_broker> [puerto_web]`	`SOCKET_PORT` (5000), `KAFKA_BROKER`, `PORT` o `WEB_PORT` (8080), `DB_PATH`
WM_WS_M (Monitor)	`python WM_WS/WM_WS_M.py <ip_central> <puerto_central> <id_ws> <ubicacion> <puerto_engine> <kafka_broker>`	`CENTRAL_HOST`, `CENTRAL_PORT`, `WS_ID`, `WS_UBICACION`, `ENGINE_PORT` (6001), `KAFKA_BROKER`
WM_WS_E (Engine)	`python WM_WS/WM_WS_E.py <ip_monitor> <puerto_monitor> <kafka_broker>`	`MONITOR_HOST`, `MONITOR_PORT` (6001), `KAFKA_BROKER`
WM_FO	`python WM_FO/WM_FO.py <kafka_broker> <id_operador> [fichero.json]`	`KAFKA_BROKER`, `OPERADOR_ID`, `ACTIVACIONES_FILE`

El puerto del panel web de Central se toma, por orden: argumento → `PORT` (el que asigna Railway) → `WEB_PORT` → 8080.

7.2 Topics de Kafka

Topic	Quién escribe → quién lee	Contenido
`telemetria-riego`	Engine → Central	Caudal y volumen cada segundo.
`comandos-estacion`	Central → Engine	Iniciar/detener riego, bloquear, activar.
`estado-ws`	Monitor → Central	Fugas, desconexiones del Engine.
`riego-peticiones`	FO → Central	Petición de riego de un operario.
`riego-respuestas`	Central → FO	Autorizado, denegado, interrumpido, resumen.
`central-broadcast`	Central → FO	Estado de todas las estaciones (cada 2 s).

7.3 Protocolo de sockets

Los mensajes por socket usan el empaquetado `<STX><DATOS><ETX><LRC>`:

- `<DATOS>` tiene la forma `OPERACION#campo1#campo2…`, por ejemplo `REGISTRO#WS-01#River Park`.
- `<LRC>` es el XOR de todos los bytes del mensaje, para detectar corrupción. El receptor responde `<ACK>` si es correcto o `<NACK>` si no.
- Cada conversación empieza con `<ENQ>` ("¿estás ahí?") que se responde con `<ACK>`.
- El Monitor mantiene la conexión con Central abierta y envía un `<ENQ>` cada 5 s como latido. Si Central no recibe nada en 15 s, o la conexión se cierra, marca la estación como **DESCONECTADA**.

El código está en `utils/protocol.py`.

7.4 Base de datos

Central usa **SQLite** (un único archivo, `wm_central.db`; en Railway, `/app/data/wm_central.db` dentro del volumen). Guarda las estaciones (id, ubicación, estado, caudal, volumen, operario), los operarios y los eventos del sistema. El código está en `utils/db.py`.